Multi-Agent Trajectory

Algorithms in optimization

Students:
Idan Feygelman
Ori Rami Eliyahu
Guy Zahav
Lecturer:
Prof. Aviv Gibali
25 June 2026

Introduction & problem statement

The goal of this project is to navigate multiple autonomous robots through a shared 3D space from their individual starting positions to their unique final destinations. The system must calculate a complete flight path for every single robot simultaneously. However, because the operational arena is shared, robots might collide with each other, and there are static obstacles in the space that the robots might crash into. Therefore, the pathfinding framework must ensure that each robot finds a path that does not crash into an obstacle nor conflict with other robots.

To evaluate the mathematical and computational trade-offs inherent in multi-agent pathfinding (MAPF). This report introduces and contrasts five distinct algorithmic approaches structured across a progressive architectural spectrum:

1. Model 1: Conflict-Based Search (CBS) - A discrete combinatorial baseline exploring a spatio-temporal constraint tree to guarantee absolute global path optimality on a grid.
2. Model 2: Decoupled Prioritized Planning - A sequential continuous optimization method optimizing agents one by one on a strict priority hierarchy.
3. Model 3: Centralized Sequential Convex Programing (SCP) - A mathematically rigorous hybrid balancing time complexity and kinetic smoothness via simultaneous local linearization.
4. Model 4: Artificial Potential Fields (APF) - A continuous, localized reactive approach prioritizing instantaneous execution speed.
5. Model 5: Simulated Annealing (SA) - A stochastic metaheuristic optimization framework designed to escape local minima traps.

Mathematical modeling

General foundations

The Physical workspace of the multi-agent system is modeled as a continuous, bounded two-dimensional real vector space: $w \subset \mathbb{R}^3$

Let N be the total number of autonomous robots, indexed by

$i \in \{1, 2, \ \dots \ N\}$

We discretize the continuous time horizon into T uniform time steps indexed by $t \in \{1, 2, \ \dots \ T\}$

The spatial position of a single robot i at a specific time step t is represented a 3D coordinate vector:

$x_{i,t} = (x \ \ y \ \ z)^T \in \mathbb{R}^3$

The complete trajectory an individual robot i over the entire space station represented by a matrix X_i:

$X_i = (x_{i,1} \ x_{i,2} \ \dots \ x_{i,T}) \in \mathbb{R}^{3\times T}$

The state of the entire system is the collection of all individual trajectories:

$X = \begin{pmatrix} X_1 & X_2 & \dots & X_N \end{pmatrix} \in \mathbb{R}^{3\times NT}$

The Objective Function

The primary optimization objective is to minimize the total trajectory length for all robots simultaneously. Minimizing the sum squared Euclidean distances between sequential time steps acts as a proxy for minimizing kinetic energy and ensuring smooth velocity transitions across the entire fleet. The global cost function $F(X)$ is formulated as:

$$\min_{X} F(X) = \sum_{i=1}^{N} \sum_{t=1}^{T-1} \|x_{i,t+1} - x_{i,t}\|_2^2$$

Boundary and safety constraints

To satisfy basic navigation requirements, each individual trajectory matrix must be strictly anchored to its designated physical start coordinate s_i and target destination coordinate d_i:

$$x_{i,1} = s_i \quad x_{i,T} = d_i \quad \forall i \in \{1, \ldots, N\}$$

1. Static Obstacles Avoidance

Let there be M static obstacles in the environment, indexed by $j \in \{1, 2, \ldots M\}$. Each obstacle is represented by its continuous Signed Distance Function (SDF), denoted as $sd_j(x) : \mathbb{R}^3 \to \mathbb{R}$.

The functional $sd_j(x_{i,t})$ outputs the shortest Euclidean distance from the robots centre coordinate $x_{i,t}$ to the closest boundary surface of the obstacle j. The metric scales such that:

$sd_j(x) < 0$ defines the forbidden volume inside the obstacle

$sd_j(x) = 0$ defines the exact boundary surface of the obstacle

$sd_j(x) > 0$ defines the safe outer space

To ensure that the physical body of the robot never collides with a static wall, the distance from its centre to the surface must be strictly greater than its radius ($sd_j(x_{i,t}) > r_i$). To satisfy standard optimization conventions requiring closed feasible set ($g(x) \leq 0$) to guarantee convergence, we incorporate a conservative safety margin buffer $\varepsilon > 0$ and formulate the non-strict inequality constraint as:

$$r_i + \varepsilon - sd_j(x_{i,t}) \leq 0$$

$$\forall i \in \{1,\ldots,N\} \quad \forall j \in \{1,\ldots,M\} \quad \forall t \in \{1,\ldots,T\}$$

2. Inter-Robot Collision Avoidance

To prevent collision between the robots, the physical footprint of any two unique robots (i and k) must never overlap at any shared fraction of time t. The euclidean distance between their centre coordinates must remain greater than the sum of their combined radiuses.

For non-strict inequality, we introduce a another buffer variable $\delta > 0$.

The constraint is formulated as:

$$\|x_{i,t} - x_{k,t}\|_2 \geq r_i + r_k + \delta$$

$$(r_i + r_k + \delta)^2 - \|x_{i,t} - x_{k,t}\|_2^2 \leq 0$$

$\forall t \in \{1,\ldots,T\} \quad \forall i,k \in \{1,\ldots,N\}$ where $i \neq k$

Model 1: Conflict-Based Search (CBS) - The Discrete, Path-Optimal Formulation

To establish a baseline that guarantees absolute global path-optimality, CBS maps the continuous workspace into a discrete topological graph $G = (V, E)$, where V represents passable grid squares and E represents valid directional steps.

The state of agent i at time step t is restricted to a discrete space-time tuple:

$$s_{i,t} = (v, t) \in V \times \mathbb{N}$$

Let C_i be a set of spatio-temporal constraints imposed on agent i by a high level coordinator. The low level path planner minimizes individual arrival steps T_i :

$$\min T_i$$

Subject to: $s_{i,1} = (s_i, 1) \quad s_{i,T_i} = (d_i, T_i) \quad (s_{i,t}, s_{i,t+1}) \in E \quad s_{i,t} \notin C_i$

A^* Low Level Planner

To compute paths within a discrete graph under dynamic multi-agent constraints, the low-level planner utilizes a Spatio-Temporal A^* search. In a 3D workspace, each vertex $v \in V$ represents a discrete volumetric coordinate (x, y, z). To prevent collisions across time, the search space is expanded into a 4D state tuple: $s = (x, y, z, t) \in V \times \mathbb{N}$

At any given time step t, an agent evaluating its neighbors from state $s_t = (x, y, z, t)$ transitions to adjacent vertices on the next time step $t + 1$. In a standard 3D grid topology, the planner evaluates a 26-way Spatial Movement, allowing the agent to transition to any adjacent voxel in its immediate 3D neighborhood where $\Delta x, \Delta y, \Delta z \in \{-1, 0, 1\} \quad (\Delta x, \Delta y, \Delta z) \neq (0, 0, 0)$. Every valid spatial transition increments the temporal component by exactly +1.

Open Set, Closed Set, and Total Cost Formulation

The low-level search manages its path expansion by partitioning the space-time state space into two core operational data structures:

1. The Open Set (Frontier): A priority queue containing all discovered space-time states that have not yet been fully evaluated. Nodes in this frontier are order strictly by their total estimated cost $f(s)$.
2. The Closed Set (Explored): A lookup set containing all space-time states that have already been popped and fully evaluated. Moving a vertex state from Open Set into the Closed Set locks that specify state; once a state $s = (x, y, z, t)$ enters the Closed Set, it is never re-examined, preventing from infinite loops and ensuring algorithm convergences.

To determine which path branch to expand from the Open Set, the independent planner evaluates states using the fundamental $A*$ cost equation: $f(s) = g(s) + h(s)$ where:

$g(s)$ is the exact cost accumulated to reach the current space-time state $s = (s, y, z, t)$ from the starting configuration. Since grid moves occur over a uniform intervals, each valid spatial progression increments $g(s)$ by exactly 1.

$h(s)$ is the estimated cost remaining to reach the destination voxel. To guarantee absolute global path optimality, the heuristic function optimality $h(s)$ must remain strictly admissible (never overestimating the true remaining cost) and consistent.

For a 3D grid topology, the planner calculates $h(s)$ as the true geometric Euclidean distance to the target destination coordinate $d_i = (x_d, y_d, z_d)$, completely oblivious to other agents:

$$h(s) = d_2(s, d_i) = \sqrt{(x - x_d)^2 + (y - y_d)^2 + (z - z_d)^2}$$

The node with the lowest combined $f(s)$ score is popped from the Open Set priority queue first. If its spatial component matches d_i, the path is reconstructed, and the search terminates successfully.

Optimal Path Reconstruction via Backtracking

When the spatial component of a popped state $s_{current}$ matches the destination coordinates d_i, the search phase terminates.

However, because $A*$ explores nodes forward, the final state does not intrinsically contain the ordered sequence of positions. To generate the actual path matrix, the algorithm must reconstruct the trajectory in reverse. This is achieved using the $CameFrom$ data structure, which functions as a directed pointer map. Every time a candidate state s_{next} is successfully validated and added to the Open Set, the algorithm registers a pointer: $CameFrom[s_{next}] = s_{current}$. Because each state has exactly one parent node that yielded its optimal g-score, backtracking from the destination guaranteed by the admissibility of $h(s)$ creates a deterministic, loop-free path backwards. The reconstruction pipeline operates as follows:

1. Initialization: An empty sequence is initialized, and the pointer is set to the terminal destination state $s_{target} = (d_i, T_i)$.
2. Reverse Traversal: The algorithm iteratively queries the pointer map: $s \leftarrow CameFrom[s]$. Each state is prepended to the sequence until the temporal component counts back down to $t = 1$, indicating the starting configuration $s_{start} = (s_i, 1)$ has been reached.
3. Dimensional Extraction: The temporal components are stripped, leaving an ordered sequence of 3D spatial coordinates. This final array is handed back to the high-level coordinator as the agents optimal individual trajectory: $\tau_i = \left[s_i, \ldots, d_i\right]^T \in \mathbb{R}^{3\times T_i}$

Here is a pseudocode of $A*$ algorithm (High-Level only):

```
Function Space_Time_A_Star(Start_3D, Goal_3D, Static_Obstacles,
Constraints):
    Open_Set = PriorityQueue()  # Discovered frontier sorted by f(s)
    Closed_Set = Empty Set      # Fully evaluated states (moved from open set)
    Came_From = Empty Map       # Path reconstruction tracking

    Start_State = (Start_3D, t=1)
    g_score[Start_State] = 0
    f_score[Start_State] = Heuristic_3D(Start_3D, Goal_3D)

    Open_Set.Insert(Start_State, f_score[Start_State])

    While Open_Set is not Empty:
        Current_State = Open_Set.PopMin()  # s = (x, y, z, t)
        Current_Pos, t_curr = Current_State.pos, Current_State.t

        If Current_Pos == Goal_3D:
            Return Reconstruct_Path(Came_From, Current_State)

        Closed_Set.Add(Current_State)  # Vertex moved to closed set

        # Generate 26 spatial neighbors
        Neighbors = Get_3D_Grid_Neighbors(Current_Pos)

        For each Next_Pos in Neighbors:
            t_next = t_curr + 1
            Candidate_State = (Next_Pos, t_next)
```

```
        # 1. Prune if already evaluated in Closed Set
        If Candidate_State in Closed_Set: Continue

        # 2. Prune via static obstacle voxels (SDF check)
        If Is_Inside_Obstacle(Next_Pos, Static_Obstacles): Continue

        # 3. Prune via spatio-temporal constraint table violations (C_i)
        If Is_Violating_Constraints(Current_Pos, Next_Pos, t_curr, Constraints):
Continue

        Tentative_g = g_score[Current_State] + 1  # Uniform step cost

        If Tentative_g < g_score[Candidate_State]:
            Came_From[Candidate_State] = Current_State
            g_score[Candidate_State] = Tentative_g
            f_score[Candidate_State] = Tentative_g + Heuristic_3D(Next_Pos,
Goal_3D)

            If Candidate_State not in Open_Set:
                Open_Set.Insert(Candidate_State, f_score[Candidate_State])

    Return Empty_Path  # Infeasible path selection branch
```

The high-level search plays all individual trajectories forward on a shared timeline. If two agents attempt to occupy the same cell or cross the same edge in opposite directions simultaneously, a logical conflict is thrown:

$$s_{i,t} == s_{k,t} \implies Conflict\ C = (i, k, v, t)$$

The high-level search resolves this conflict by branching a Constraint Tree (CT) into two child nodes, separating the non-convex options into discrete traffic constraints:

Left child: Appends the constraint (v, t) to agent i ($C_i \leftarrow C_i \cup \{(v, t)\}$).

Right child: Appends the constraint (v, k) to agent k ($C_k \leftarrow C_k \cup \{(v, t)\}$).

The tree is explored via a Priority Queue sorted by the sum of individual costs ($\sum T_i$). Whichever detour branch prolonged the total travel time the least bubbles to the top, guaranteeing that the first conflict-free leaf node popped is globally path-optimal.

Convergence

Spatio-Temporal Discretization Bridge

To resolve the contradiction of using a discrete graph search tool alongside continuous optimization frameworks, a finite spatial discretization mesh of resolution Δx is applied. An arbitrary continuous obstacle geometry is mapped to the discrete graph using a conservative boundary evaluation rule:

if $\exists x \in Cell(v)$ such that $sd_j(x) \leq r_i + \varepsilon \implies v \in \mathcal{O}_{static}$

This transforms continuous boundary penetration into discrete vertex/ edge constraints, validating the theoretical baseline comparison.

Because CBS operates on a discrete patio-temporal graph mesh $G = (V, E)$ rather than a continuous vector space, its convergence properties are combinatorial rather than differential.

Theorem

For a bounded discrete graph workspace containing a finite fleet size N over a maximum path timeline T, a high-level Constraint Tree (CT) search will terminate in a finite number of steps, converging exactly to the absolute globally optimal conflict-free solution, provided a valid solution path exists.

Proof via Branching Bounds

Let the low-level space-time $A*$ search be complete and optimal on the discrete graph mesh. The high-level search explores a tree where each node represents a specific set of binary multi-agent conflict constraints $C_i = (i, k, v, t)$.

1. The total number of unique spatio-temporal conflict pairs (v, t) within a bounded vertex set V and a maximum timeline horizon T is strictly finite: $|V| \times T$.
2. Because each high-level branching action adds at least one mandatory constraint to a specific agent $(C_i \leftarrow C_i \cup \{(v, t)\})$, it permanently prunes that specific conflicting path option conflicting path option from the low-level search space.
3. Since the underlying graph is finite, the maximum depth of the Constraint Tree is bounded. Because the tree is explored using a Priority Queue order monotonically by the total path cost ($\sum T_i$), the first left node popped that contains zero internal collisions is mathematically guaranteed to be the global minimum-cost routing configuration.
4. Convergence Rate: Structurally exponential in the worst-case ($\mathcal{O}(2^C)$, where C is the total number of resolved conflicts), exhibiting a steep combinatorial complexity trade-off to ensure absolute global optimality.

Here is a pseudocode for CBS algorithm:

```
Function Conflict_Based_Search(Starts, Destinations, Obstacles):
    Root = New CT_Node(Constraints = Empty)
    For i = 1 To N:
        Root.Paths[i] = Space_Time_A_Star(Starts[i], Destinations[i], Obstacles,
Root.Constraints[i])
        If Root.Paths[i] is Empty: Return Fail

    Open_CT = PriorityQueue(Root sorted by Sum_of_Costs)

    While Open_CT is not Empty:
        Current = Open_CT.PopMin()
        Conflict = Validate_All_Paths_Timeline(Current.Paths)

        If Conflict is None: Return Current.Paths  # Globally optimal solution

        # Conflict: C = (Agent_i, Agent_k, Voxel_v, Time_t)
        For Agent in [Agent_i, Agent_k]:
            Child = New CT_Node(Constraints = Copy(Current.Constraints), Paths =
Copy(Current.Paths))
            Child.Constraints[Agent].Add((Voxel_v, Time_t))

            Child.Paths[Agent] = Space_Time_A_Star(Starts[Agent],
Destinations[Agent], Obstacles, Child.Constraints[Agent])

            If Child.Paths[Agent] is not Empty:
                Open_CT.Insert(Child, Get_Sum_Of_Costs(Child.Paths))
    Return Fail
```

Model 2: Decoupled Prioritized Planning - The Greedy, Linear-Time Formulation

To drastically reduce computational complexity, Model 2 preserves continuous coordinate optimization but completely decouples the multi agent system into an isolated, sequential pipeline.

The fleet is sorted into a strict hierarchy (e.g. Agent 1 has the highest priority, while Agent N has the lowest). Robots optimize their paths one by one. When it is Agent i's turn to plan, the trajectories of all higher-priority agents $k < i$ have already calculated and locked down.

Instead of treating teammates as active optimization variables, Agent i treats them as predictable, fixed time-varying obstacle constants, noted as $\bar{x}_{k,t}$. The decoupled optimization problem for Agent i simplifies to:

$$\min_{X_i} \sum_{i=1}^{T-1} \|x_{i,t+1} - x_{i,t}\|_2^2$$

Subject to:

$$r_i + \varepsilon - sd_j(x_{i,t}) \leq 0 \quad \forall j, t$$

$$(r_i + r_k + \delta)^2 - \|x_{i,t} - \bar{x}_{k,t}\|_2^2 \leq 0 \quad \forall t \ \forall k < i$$

$$x_{i,1} = s_i \quad x_{i,T} = d_i$$

Because each agent is optimized completely independently, the total state-space scales linearly ($O(N)$), ensuring exceptionally fast computational execution times. However, it is highly sub-optimal and greedy, forcing lower-priority agents to execute massive, inefficient detours to yield to higher-priority paths.

Convergence

In a prioritized planning framework, the global multi-agent problem is broken into a sequence of N separate, single-agent optimization subproblems.

Convergence Mechanics

The global fleet convergence is completely dependent on the successful completion of the single-agent pipeline cascade. Each individual agent optimizes its own path vector X_i sequentially while treating all previously calculated higher-priority agents paths as a static spatio-temporal obstacles

$$\bar{x}_{k,t}: \min_{X_i} \sum_{t=1}^{T-1} \|x_{i,t+1} - x_{i,t}\|_2^2$$

1. Rate: If each decoupled subproblem is solved using a convex solver or a continuous interior-point optimizer, each individual agent achieves polynomial-time or linear convergence to its restricted subproblem minimum.
2. Global Completeness Flaw: There is no global convergence guarantee for the fleet. If a high-priority agent locks down a path that physically blocks a narrow corridor, the feasible set for lower-priority agents becomes completely empty ($\mathcal{F}_i = \varnothing$). The sequential loop fails to find a valid mathematical answer, resulting in an algorithmic deadlock.

Here is a pseudocode for Decoupled Algorithm:

```
Function Decoupled_Prioritized_Planning(Agents, Priorities, Obstacles):
    Sorted_Agents = Sort(Agents, Key=Priorities, Order=Descending)
    Locked_Paths = Empty Map

    For each Agent i in Sorted_Agents:
        Problem = New TrajectoryOptimizationProblem(Start = s_i, Goal = d_i)
        Problem.AddStaticConstraints(Obstacles)

        # Treat higher priority paths as dynamic obstacle constants
        For each Path in Locked_Paths.Values():
            Problem.AddDynamicObstacleConstraint(Path, Safety_Buffer)

        Trajectory_i = Minimize_Sum_Squared_Distances_Solver(Problem)
        If Trajectory_i is Infeasible: Return Fail  # Priority deadlock

        Locked_Paths[Agent_i] = Trajectory_i

    Return Locked_Paths
```

Model 3: Centralized Sequential Convex Programming (SCP) - The optimal continuous hybrid

Model 3 provides the ultimate balance by optimizing all continuous robot trajectories simultaneously. To handle non-convex constraints without incurring exponential delays, SCP takes the raw physical equations and linearizes them locally using first-order Taylor series approximations.

At each major loop iteration m, the algorithm utilizes a reference trajectory guess $\bar{X}^{(m)}$ obtained from the previous iteration. The non-convex constraints are rewritten as flat linear hyperplanes.

Geometric pre-conditioning via Minkowski Sum Inflation:

To treat each autonomous agent i as an infinitely small point max vector $x_{i,t} \in \mathbb{R}^3$ and eliminate the gardient discontinuities caused by sharp-edged environmental obstacles, the workspace undergoes a Minkowski sum configuration space ($\mathscr{C}$ space) transformation. Because each agent is modeled as a perfect rigid spherical ball B_i of radius r_i centered at its origin, the Minkowski sum with a static obstacle geometry set $\mathcal{O}_j$ is defined as:

$$\mathcal{O}_{j,inflated} = \mathcal{O}_j \oplus B_i = \{a + b \mid a \in \mathcal{O}_j \wedge b \in B_i\}$$

Since the robot geometry B_i is a perfect sphere, sliding it along the perimeter of the obstacle acts as a uniform geometric dilation. This operation automatically shifts flat obstacle walls outward by exactly r_i and transforms sharp, non-differentiable vertices into perfectly smooth spherical arcs of radius r_i. The continuous Signed Distance Function $sd_j(x_{i,t})$ is subsequently queried directly against this smoothed, pre-inflated geometric set $\mathcal{O}_{j,inflated}$, providing continuous, well-defined gradients ∇sd_j across the entire boundary.

Convexifying Curved Obstacle Boundaries

The curved obstacle boundaries are flattened into a straight line tangent to the closest surface normal vector ∇sd_j:

$$(r_i + \varepsilon) - sd_j(\bar{x}_{i,t}^{(m)}) - \nabla sd_j(\bar{x}_{i,t}^{(m)})^T(x_{i,t} - \bar{x}_{i,t}^{(m)}) \leq 0$$

The inter-robot distance constraint is linearized around the reference coordinates $\bar{x}_{i,t}^{(m)}$ and $\bar{x}_{k,t}^{(m)}$. Let $D_{ik} = \|\bar{x}_{i,t}^{(m)} - \bar{x}_{k,t}^{(m)}\|_2$. The non-convex distance ball is mapped to a convex linear hyperplane inequality:

$$(r_i + r_k + \delta) - D_{ik} - \left(\frac{\bar{x}_{i,t}^{(m)} - \bar{x}_{k,t}^{(m)}}{D_{ik}} \right) \left((x_{i,t} - \bar{x}_{i,t}^{(m)}) - (x_{k,t} - \bar{x}_{k,t}^{(m)}) \right) \leq 0$$

Trust-Region Formulation

To prevent the optimizer from making large spatial jumps where the first-order Taylor series approximation breaks down, a hard trust-region bounding box constraint is appended to the subproblem:

$$\|x_{i,t} - \bar{x}_{i,t}^{(m)}\|_\infty \leq \Delta^{(m)}$$

Because the master objective function is quadratic and all linearized safety constraints are convex inequalities, the problem simplifies at iteration m into a standard Convex Quadratic Program (QP). This is solved concurrently for all agents in polynomial time ($O(N^3)$).

Convergence

SCP solves a sequence of convex Quadratic Programs (QPs) Generated by substituting non-convex obstacle bounds with local first-order Taylor series expansion around a reference Trajectory $\bar{X}^{(m)}$.

Theorem (Convergence to Karush-Kuhn-Tucker Points)

Mathematical Elaboration of the KKT Conditions for Centralized SCP

To formally establish local optimality, the optimal state matrix $X^* = \left[X_1^*, X_2^*, \ldots, X_N^*\right]$ must satisfy the first-order necessary conditions for the original non-convex optimization problem. Let $\lambda_{ij,t} \geq 0$ be the Lagrange multipliers associated with the static obstacle constraints, and $\gamma_{i,k,t} \geq 0$ be the multipliers for the non-convex inter-robot separation boundaries.

The Lagrangian function

$$\mathscr{L}(X, \lambda, \gamma) = F(X) + \sum_{i=1}^{N}\sum_{j=1}^{M}\sum_{t=1}^{T} \lambda_{ij,t}\left(r_i + \varepsilon - sd_j(x_{i,t})\right) + \sum_{i=1}^{T}\sum_{i=1}^{N}\sum_{k<i} \gamma_{ik,t}\left((r_i + r_k + \delta)^2 - \|x_{i,t} - x_{k,t}\|_2^2\right)$$

For the iterative sequence $\{X^{(m)}\}$ generated by the SCP framework to converge to a true local minimum X^*, the final limit point must satisfy the four core blocks of the Karush-Kuhn-Tucker (KKT) system:

1. Primal Feasibility (Strict Collision Avoidance)

The trajectory variables must reside inside the intersecting boundaries of the non-convex safe operational set $\mathbb{F}$:

$$r_i + \varepsilon - sd_j(x_{i,t}^*) \leq 0 \quad \forall i, j, t$$

$$(r_i + r_k + \delta)^2 - \|x_{i,t}^* - x_{k,t}^*\|_2^2 \leq 0 \quad \forall i, (k < i), t$$

2. Dual Feasibility

The shadow prices (Lagrange multipliers) capturing constraint sensitivity must remain non-negative, meaning constraints can only push the robots away from forbidden boundaries, never pull them in:

$$\lambda_{ij,t} \geq 0 \quad \gamma_{ik,t} \geq 0$$

3. Complementary Slackness

Multipliers are strictly decoupled from inactive constraints. If an agent is navigating through completely wide-open space where the distance buffers are strictly positive, the corresponding optimization multipliers collapse identically to zero:

$$\lambda_{ij,t} \cdot \left(r_i + \varepsilon - sd_j(x^*_{i,t}) \right) = 0 \quad \forall i, j, t$$

$$\gamma_{ik,t} \cdot \left((r_i + r_k + \delta)^2 - \|x^*_{i,t} - x^*_{k,t}\|_2^2 \right) = 0 \quad \forall i, (k < i), t$$

4. Stationarity (Vanishing Lagrangian Gradient)

The total gradient of the objective function (the lease-squares kinetic energy minimization) must be perfectly counterbalanced by the linear combination of the active geometric constraint gradients at the KKT point:

$$\nabla_X F(X^*) + \sum_{i,j,t} \lambda_{ij,t} \left(-\nabla_X sd_j(x^*_{i,t}) \right) + \sum_{t,i,(k<i)} \gamma_{ik,t} \left(-2(x^*_{i,t} - x^*_{k,t}) \right) = 0$$

Let $X^{(m)}$ be the optimal solution vector obtained at major loop iteration m. If the trust-region constraint parameter is dynamically updated according according to a standard filtering method: $\|X - \bar{X}^{(m)}\|_\infty \leq \Delta^{(m)}$

And the objective function is bounded from below, the iterative sequence $\{X^{(m)}\}$ is guaranteed to converge asymptotically to a first-order local optimum, satisfying the exact Karush-Kuhn-Tucker (KKT) conditions of the original non-convex problem.

Dynamic Radius Adaptation via Step-Quality Filtering

To regulate convergence, the trust-region radius is updated dynamically at the end of each iteration by evaluating the local fidelity of linearization. This is achieved via the trust-region quality metric ρ_m, which computes the ratio between the actual reduction in the non-linear cost function and the reduction predicted by the convex subproblem:

$$\rho_m = \frac{F(X^{(m)}) - F(X^{(m+1)})}{F(X^{(m)}) - \hat{F}^{(m)}(X^{(m+1)})}$$

Where $\hat{F}^{(m)}$ represents the local convex approximation of the system state landscape.

The adaptation loop executes the following structural logic path:

Case 1: $\rho_m < 0.25$ tight squeeze/ high non-linearity

The linear approximation deviates significantly from the true landscape geometry (e.g. navigating near a highly curved obstacle boundary or sharp corner). The candidate step is rejected, the robot position remains unchanged $\bar{X}^{(m+1)} = \bar{X}^{(m)}$, and the trust region is contracted to enforce higher approximation fidelity:

$\Delta^{(m+1)} = \beta_{shrink} \cdot \Delta^{(m)}$ where $\beta_{shrink} \in (0,1)$

Case 2: $0.25 \leq \rho_m \leq 0.75$ steady-state tracking

The prediction matches reality reasonably well. The step is a accepted, the trajectory updates, and the radius is held constant $\Delta^{(m+1)} = \Delta^{(m)}$

Case 3: $\rho_m > 0.75$ wide-open expansion

The workspace environment matches the flat prediction perfectly (e.g. traveling along a straight projector in the safe zone). The step is accepted, and the trust-region box expands to accelerate convergence speed:

$\Delta^{(m+1)} = \min(\gamma_{grow} \cdot \Delta^{(m)}, \Delta_{max})$ where $\gamma_{grow} > 1$

Convergence Rate Derivation

Because SCP mimics the structural mechanics of a generalized Sequential Quadratic Programming (SQP) algorithm under a trust region, its convergence rate is fundamentally governed by the approximation behavior of the linearized constraints.

1. Let $e^{(m)} = \|X^{(m)} - X^*\|_2$ represent the exact distance error to the local KKT point X^*.
2. By utilizing exact analytical first-order gradients (∇sd_j) for the Taylor expansion rather than loose numerical finite differences, the Taylor remainder theorem bounds the linear residual error quadratically: $\|g(X) - g_{linear}(X)\| \leq \frac{1}{2} L \|X - \bar{X}^{(m)}\|_2^2$ Where L is Lipschitz constant of the obstacle gradient field.
3. This structural property guarantees that the error sequence satisfies: $\lim_{m \to \infty} \frac{e^{(m+1)}}{(e^{(m)})^2} = C < \infty$ Hence, inside a valid trust-region radius,

   Centralized SCP exhibits a quadratic convergence rate $\mathcal{O}((e^{(m)})^2)$, enabling it to locate smooth, kinetically optimal trajectories in very few iterations.

Here is a pseudocode for SCP:

```
Function Centralized_SCP(Starts, Goals, Obstacles):
    X_Ref = Initialize_Straight_Line_Trajectories(Starts, Goals)
    Delta = Initial_Trust_Region_Radius
    For m = 1 To Max_Iterations:
        QP = New QuadraticProgram(Objective = Minimize_Global_Kinetic_Cost)
        QP.AddBoundaryConditions(Starts, Goals)

        # 1. First-order Taylor linearization of non-convex environmental
obstacles
        QP.AddConstraints(Linearize_SDF(Obstacles, X_Ref))
        # 2. First-order Taylor linearization of non-convex inter-agent safety
spheres
        QP.AddConstraints(Linearize_InterRobot_Distances(X_Ref))
        # 3. Trust region boundary box constraints
        QP.AddTrustRegionConstraint(X_Ref, Delta)

        X_Candidate = Solve_Convex_QP(QP)
        Rho = Get_Actual_Reduction(X_Candidate) /
Get_Predicted_Reduction(X_Candidate)

        If Rho < 0.25:
            Delta = Beta_Shrink * Delta  # Step rejected, contract region
        Else:
            X_Old = X_Ref; X_Ref = X_Candidate  # Step accepted
            If Rho > 0.75: Delta = Min(Gamma_Grow * Delta, Delta_Max)

        If Norm(X_Ref - X_Old) < Convergence_Tolerance: Return X_Ref
    Return X_Ref
```

Model 4: Artificial Potential Fields (APF) - Reactive Continuity

Model 4 treats each autonomous robot as a charged particle navigating continuous virtual potential energy field. The destination coordinate exerts an attractive potential pulling the robot forward, while static obstacles and surrounding agents exert repulsive potentials pushing the robot away.

The potential field $U(x_{i,t})$ acting on robot i at time step t is the sum of its attractive field U_{att} and repulsive fields U_{rep}:

$$U(x_{i,t}) = U_{att}(x_{i,t}) + \sum_{j=1}^{M} U_{rep,obs}(x_{i,t})^{(j)} + \sum_{k\neq j}^{N} U_{rep,rob}(x_{i,t})^{(k)}$$

The attractive potential pulls the agent toward its destination coordinate d_i:

$$U_{att} = \frac{1}{2} k_{att} \|x_{i,t} - d_i\|_2^2$$

Taking the spatial derivatives with respect to the 3D coordinate vector components yields:

$$\nabla U_{att} = k_{att}(x_{i,t} - d_i)$$

Dynamic Inter-robot Repulsive Potential ($U_{rep,rob}$)

To maintain a safe separation envelope between active agents, a distinct dynamic repulsive potential is formulated. Because both interacting bodies are rigid circular disks, the clear distance between them depends on the euclidean distance separating their centers minus their combined physical radii $(r_i + r_k)$. It activates when teammates venture within an influence buffer ρ_{rob}:

$$U_{rep,rob}^{(k)}(x_{i,t}) = \begin{cases} \frac{1}{2} k_{rob} \left(\frac{1}{\|x_{i,t} - x_{k,t}\|_2 - (r_i + r_k + \delta)} - \frac{1}{\rho_{rob}} \right)^2 & \text{if } \|x_{i,t} - x_{k,t}\|_2 - (r_i + r_k) \leq \rho_{rob} + \delta \\ 0 & \text{if } \|x_{i,t} - x_{k,t}\|_2 - (r_i + r_k) > \rho_{rob} + \delta \end{cases}$$

$$F_{rep,rob}^{(k)}(x_{i,t}) = -\nabla U_{rep,rob}^{(k)}(x_{i,t}) = k_{rob} \left(\frac{1}{\|x_{i,t} - x_{k,t}\|_2 - (r_i + r_k + \delta)} - \frac{1}{\rho_{rob}} \right) \frac{1}{(\|x_{i,t} - x_{k,t}\|_2 - (r_i + r_k + \delta))^2} \cdot \left(\frac{x_{i,t} - x_{k,t}}{\|x_{i,t} - x_{k,t}\|_2} \right)$$

Where $k_{rob} > 0$ represents the active multi-agent evasive tracking gain.

Static Obstacle Repulsive Potential ($U_{rep,obs}$)

The repulsive potential shielding the fleet from fixed environmental boundaries uses the continuous Signed Distance Function $sd_j(x_{i,t})$. It isolates the boundary and spikes toward infinity as the distance approaches zero, activating only within a local influence threshold ρ_{obs}:

$$U^{(j)}_{rep,obs}(x_{i,t}) = \begin{cases} \frac{1}{2}k_{obs}\left(\frac{1}{sd_j(x_{i,t}) - r_i - \varepsilon} - \frac{1}{\rho_0}\right)^2 & \text{if } sd_j(x_{i,t}) \le \rho_0 + r_i + \varepsilon \\ 0 & \text{if } sd_j(x_{i,t}) > \rho_0 + r_i + \varepsilon \end{cases}$$

$$F^{(j)}_{rep,obs}(x_{i,t}) = -\nabla U^{(j)}_{rep,obs}(x_{i,t}) = k_{obs}\left(\frac{1}{sd_j(x_{i,t}) - r_i - \varepsilon} - \frac{1}{\rho_{obs}}\right)\frac{1}{(sd_j(x_{i,t}) - r_i - \varepsilon)^2} \cdot \nabla sd_j(x_{i,t})$$

Where $\nabla sd_j(x_{i,t})$ is the outward-pointing unit normal vector form the obstacle surface toward the robot.

$k_{obs} > 0$ is the static barrier scaling parameter.

Analytical Derivation of the Control Law via Vector Gradients

To translate these scalar fields into a physical steering command vector $F_{total}(x_{i,t})$, we compute the negative spatial gradient of the combined potential field ($F_{total} = -\nabla U$). Applying the multivariable chain rule to the independent components yields the complete control law:

$$\nabla U(x_{i,t}) = F_{att}(x_{i,t}) + F_{rep}(x_{i,t})$$

Convergence

Because APF operates as a continuous reactive controller driven by the spatial gradient of a potential function $U(x)$, its motion equations match a continuous-time gradient descent system: $\dot{x}_i(t) = -\nabla U(x_i(t))$

Theorem (Convergence to Stationary Points)

If the total energy function $U(x) : \mathbb{R}^3 \to \mathbb{R}$ is a continuously differentiable $U \in \mathscr{C}^1$ and radially unbounded, the trajectory of the agent will converge asymptotically to a stationary point where the net force vector satisfies:

$$\lim_{t \to \infty} \nabla U(x_i(t)) = 0$$

Proof via Lyapunov Stability

Define the candidate Lyapunov function as the total potential energy itself: $V(x) = U(x) - U^* \geq 0$, where U^* is the global minimum energy. Taking the time derivative of $V(x)$ along the system trajectories yields:

$$\dot{V}(x) = \nabla U(x)^T \dot{x} = \nabla U(x)^T (- \nabla U(x)) = -\| \nabla U(x) \|_2^2$$

Since $\dot{V}(x) \leq 0$ universally, the system is Lyapunov stable. By LaSalle's Invariance Principle, the trajectory might get trapped in a local minima where $\dot{V}(x) = 0$, which corresponds exactly to the stationary points $\nabla U(x) = 0$.

Convergence Rate & Sub-optimality

1. Rate: Locally linear $\mathcal{O}(e^{-\lambda t})$ in the immediate neighborhood of a strongly convex potential well.
2. Failure Mode: The algorithm lacks global convergence to the target. If $\nabla U_{att}(x_{i,t}) = - \nabla U_{rep}(x_{i,t})$ at a point away form the goal, the system hits a local minimum trap, freezing execution with zero velocity.

Here is a pseudocode for APF:

```
Function Artificial_Potential_Fields(Starts, Destinations, Obstacles):
   For i = 1 To N: x[i] = Starts[i]

   For step = 1 To Max_Steps:
      If All_Robots_At_Destination(x, Destinations): Return Trajectory_Logs

      For i = 1 To N:
         F_att = -Gradient_Attractive_Potential(x[i], Destinations[i])
         F_rep_obs = Sum(Compute_Obstacle_Repulsion(x[i], Obs) For Obs in
Obstacles)
         F_rep_rob = Sum(Compute_Robot_Repulsion(x[i], x[k]) For k in 1..N if k !
= i)

         F_net = F_att + F_rep_obs + F_rep_rob

         If Norm(F_net) < 1e-5 And Not_At_Goal(x[i]): Warn_Local_Minimum(i)

         x[i] = x[i] + Time_Delta * F_net
         Log_State(i, x[i], step)

   Return Trajectory_Logs
```

Model 5: Simulated Annealing (SA) - Stochastic Metaheuristic Optimization

Unlike localized or decoupled planners that evaluate paths one agent at a time, the centralized Simulated Annealing framework optimizes the entire fleet simultaneously. The optimization begins with a global initial guess matrix $X^{(0)}$ (typically straight lines connecting each agent's start s_i and goal d_i). Because these straight lines completely ignore physics, the initial configuration state is highly infeasible, containing multiple static obstacle intersections and inter-robot collisions.

Centralized Saptio-Temoral Mutations

To explore the non-convex state space without getting trapped in local minima, the algorithm perturbs the global configuration matrix at each iteration m. Rather than mutating every waypoint simultaneously, which creates chaotic, unstable paths, the algorithm applies a targeted Randomized Component Mutation:

At each iteration step, the algorithm randomly selects a single robot $i \in \{1,\ldots,N\}$ and a single intermediate time step $t \in \{2,\ldots,T-1\}$. A candidate coordinate perturbation is generated by adding zero-mean Gaussian noise to just that specific waypoint:

$$x_{i,t}^{cand} = x_{i,t}^{(m)} + \mathcal{N}(0,\sigma^2 I_3)$$

This yields a centralized candidate state matrix X^{cand}. By restricting mutations to a single localized points along the timeline, the fleet can resolve complex spatial bottlenecks smoothly, one minor adjustments at time.

The unified Global Energy (Cost) Function

To determine a collective mutation is accepted, the algorithm measures the change in total systemic energy:

$\Delta E = F(X^{cand}) - F(X^{(m)})$. the centralized cost function $F(X)$ is mathematically unified to balance path efficiency, obstacle avoidance, and inter-robot safety buffers using soft penalty multipliers ($\mu_{obs}, \mu_{rob} > 0$):

$$F(X) = \sum_{i=1}^{N}\sum_{t=1}^{T-1}\|x_{i,t+1} - x_{i,t}\|_2^2 + \mu_{obs}\sum_{i=1}^{N}\sum_{t=1}^{T}\sum_{j=1}^{M}\max\left(0, r_i + \varepsilon - sd_j(x_{i,t})\right)^2$$
$$+\mu_{rob}\sum_{t=1}^{T}\sum_{i=1}^{N}\sum_{k<i}\max\left(0,(r_i + r_k + \delta)^2 - \|x_{i,t} - x_{k,t}\|_2^2\right)^2$$

The collective Metropolis Choreography

Because the inter-robot collision term evaluates the Euclidean distance between every unique pair of agents (i and k) at every time step t, any random modification that resolves an obstacle boundary penetration but accidentally forces that robot into a teammates's path will trigger a sharp spike in global energy ($\Delta E > 0$).

In the Higher-Temperature Phase ($T_m \gg 0$):

The system has a near 100% probability of accepting positive energy changes ($\exp(-\Delta E/T_m) \to 1$). This allows the trajectories of different robots to wildly cross over, pass through one another, and temporarily clip static obstacles. This phase determines the macroscopic traffic choreography, deciding which agents go left, right, first or second through tight doorways.

In the Low-Temperature Phase ($T_m \to 0$):

As the geometric cooling schedule progresses ($T_m = T_0 \cdot \alpha^m$), the probability of accepting a collision or a longer exploring alternative drops to zero. The algorithm stops exploring alternative routes and freezes the system into its discovered channels, polishing the rough, mutated lines into perfectly smooth, synchronized, and completely collision-free trajectories across the entire fleet.

Convergence

Simulated Annealing samples the continuous trajectory space using a discrete-time Markov Chain Monte Carlo (MCMC) transition model parameterized by a time-varying temperature parameter T_m.

Theorem (Asymptotic Global Convergence Proof)

Let $\mathcal{X}^*$ be the set of globally optimal trajectory configurations that minimize the unified global cost function $F(X)$. If the system temperature is lowered following a geometric cooling schedule:

$$T_m = \alpha^m \cdot T_0$$

where T_0 is the initial high temperature and $\alpha \in (0,1)$ is the constant decay factor. As the iterations progress. The configuration state sequence $\{X^{(m)}\}$ satisfies: $\lim_{m \to \infty} P(X^{(m)} \in \mathcal{X}^*) = 1$

Mechanism of Geometric Relaxation

At any fixed iteration loop step m, the transition probability matrix forms a homogenous, irreducible, and aperiodic Markov chain governed by the Metropolis-Hasting criterion. The probability distribution of the system state space tracks a time-varying Gibbs-Boltzmann distribution:

$$\pi_m(X) = \frac{1}{Z(T_m)} \exp\left(-\frac{F(X)}{T_m}\right)$$

Where $Z(T_m)$ is the partition function normalization constant, summing the probabilities over all possible configuration states:

$$Z(T_m) = \int_{\Omega} \exp\left(-\frac{F(Y)}{T_m}\right) dY$$

It is important to note that because the underlying Markov chain is structurally irreducible (any valid state is reachable from any other state given sufficient time steps) and aperiodic. By maintaining the Detailed Balance condition via the Metropolis criterion, the local step dependencies fade over an infinite time horizon:

$$\pi_m(X^{(m)})P(X^{(m)} \to X^{cand}) = \pi_m(X^{cand})P(X^{cand} \to X^{(m)})$$

Taking the mathematical limit as the geometric cooling schedule drives the temperature parameter to zero $T_m \to 0$:

$$\lim_{T_m \to 0} \pi_m(X) = \frac{1}{\mu(\mathcal{X}^*)} \cdot \mathbb{I}_{\mathcal{X}^*}(X) = \begin{cases} \frac{1}{\mu(\mathcal{X}^*)} & \text{if } X \in \mathcal{X}^* \\ 0 & \text{if } X \notin \mathcal{X}^* \end{cases}$$

For a countable $\mathcal{X}^*$ it will be:

$$\lim_{T_m \to 0} \pi_m(X) = \frac{1}{|\mathcal{X}^*|} \sum_{\mathcal{X}_i^* \in \mathcal{X}^*} \delta(X - \mathcal{X}_i^*) = \begin{cases} \frac{1}{|\mathcal{X}^*|} & \text{if } X \in \mathcal{X}^* \\ 0 & \text{if } X \notin \mathcal{X}^* \end{cases}$$

Thus, the probability distribution concentrates entirely within the global minima $\mathcal{X}^*$, forcing the system to freeze into a completely optimized, collision-free routing configuration.

Convergence Rate: Linear and geometric $\mathcal{O}(\alpha^m)$, this ensures that the trajectory profiles stabilize rapidly within a finite computational horizon, delivering highly optimized, collision-free paths.

Here is a pseudocode for SA:

```
Function Centralized_Simulated_Annealing(Starts, Goals, Obstacles):
    X_Curr = Initialize_Trajectories(Starts, Goals)
    E_Curr = Compute_Unified_System_Energy(X_Curr, Obstacles)
    Temp = Initial_High_Temperature

    For m = 1 To Max_Iterations:
        X_Cand = Copy(X_Curr)
        # Apply stochastic zero-mean Gaussian perturbation onto one random waypoint
        X_Cand[Random_Agent(), Random_Time()] += Generate_Gaussian_Noise()

        E_Cand = Compute_Unified_System_Energy(X_Cand, Obstacles)
        Delta_E = E_Cand - E_Curr

        # Metropolis Acceptance Filtering
        If Delta_E < 0 Or Random_Float(0, 1) < Exp(-Delta_E / Temp):
            X_Curr = X_Cand
            E_Curr = E_Cand

        Temp = Cooling_Decay_Alpha * Temp  # Geometric relaxation schedule
        If Temp < 1e-6 And Energy_Is_Stable(): Break

    Return X_Curr
```

Comprehensive Algorithmic Comparison Matrix

The following analytical matrix summarizes the formal theoretical trade-offs derived and evaluated across all five multi-agent trajectory frameworks.

Evaluation Metric	Model 1: CBS	Model 2: Decoupled	Model 3: SCP	Model 4: APF	Model 5: SA
Optimization space	Discrete Spatio-Temporal Graph Mesh	Continuous Sequential Optimization Vector	Centralized Continuous Linearized Planes	Continuous Local Gradient Field	Continuous Stochastic Configuration
Coordination method	Hierarchical Combinatorial Tree	Sequential Priority Ranking Pipeline	Centralized Concurrent Solver Engine	Reactive Field Accumulation	Stochastic Path Displacments
Collision Handling	Spatio-Temporal Vertex/ Edge Splits	Locked Trajectory Constant Overlays	Convexified Hyperplane Separation	Repulsive Virtual Force Fields	Randomized Cost Shift filtering
Trajectory Geometry	Boxy, orthogonal Discrete Segments	Smooth Prioritized Trajectory	Centralized Smooth Aerodynamic Curves	Continuous Smooth Vector Line	Stochastic Oscillating Approximations
Optimality Class	Globally Optimal (Shortest Grid Paths)	Sub-Optimal (Greedy Path Detours)	Linearly Optimal (Kinetically Smooth)	Sub-Optimal (Vulnerable to traps)	Near-Globally Path-Optimal
Time Complexity	Exponential Time: $O(2^C)$	Linear Time: $O(N)$	Polynomial time: $O(mN^3T^3)$	Linear Time: $O(NM)$	Iterative search: $O(KN)$

Discussion: The Centralization vs Decentralization Dilemma

A core architectural dilemma arose during the development of the continuous trajectory frameworks (Models 3,4 and 5): Should the algorithms be formulated and executed as fully centralized systems, or as non-centralized (decentralized) distributed systems?

The Trade-off Matrix

Vector	**Centralized Architecture**	**Non-Centralized (Distributed) Architecture**
Mathematical Guarantees	High. The solver retains full visibility over the complete joint state-space. It guarantees convergence to an exact KKT point (SCP) or a global energy minimum, ensuring strict collision avoidance.	Reduced. Splitting the equations can cause agents to lose sight of global coupling. The system risks encountering local minima, oscillations, or uncoordinated deadlocks.
Computational Complexity	Poor. Stacking all N agents over a time horizon T causes a polynomial explosion in matrix sizes. For continuous solvers like SCP, the time complexity scales aggressively at $\mathcal{O}(mN^3T^3)$, rendering real-time scaling for large fleets computationally impossible.	Excellent. By decoupling the fleet into a sequential priority pipeline, the matrix joint problem is broken down into N isolated, single-agent subproblems. Each robot optimizes its own path independently and simultaneously updates its trajectory. This drops the fleet scaling factor to a linear profile of $\mathcal{O}(N)$, dramatically lowering the runtime footprint.

Empirical Resolution Analysis - The Sub-Optimal Gap

To evaluate the mathematical and computational trade-offs inherent in multi-agent path-finding, a benchmark simulation was conducted. This experiment evaluates the performance of Model 1: CBS across a refining spectrum of spatial grid resolutions $\Delta x \in \{2.0, 1.0, 0.5, 0.2\}$ m against Model 3: SCP, which acts as the continuous, resolution-independent baseline.

Prior to empirical testing, our theoretical expectations were twofold. First, we hypothesized that by forcing the algorithms to navigate these rigid geometric constraints, the discrete CBS path would be longer, and therefore inherently sub-optimal, compared to the continuous SCP geodesic. Second, we anticipated that attempting to close this sub-optimality gap by shrinking the spatial grid resolution (Δx) would trigger a cubic explosion in memory and computational runtime, denoted as $\mathcal{O}(1/\Delta x^3)$.

Initial Findings: The Discretization Artifact

In our initial testing using a standard center-point voxelization scheme, the benchmark revealed a severe geometric anomaly: at the finest grid resolution ($\Delta x = 0.2$ m), the CBS solver produced a path that was 1.1% shorter than the mathematically optimal SCP geodesic. This outcome is a classic manifestation of Discretization Bias, often referred to as Voxelization Aliasing. Because the grid evaluates obstacles at discrete intervals, the sharp, continuous apex of the 'L'-shaped intersection fell exactly between the evaluated voxel centers.

The CBS solver effectively found a ”micro-tunnel” that did not exist in continuous space, generating a path that was mathematically optimal for the grid, but physically in collision with the true Signed Distance Field (SDF) of the obstacle. This bug serves as a critical reminder of the risks inherent in discrete approximation schemes, where numerical efficiency can inadvertently compromise physical safety constraints.

Conservative Voxelization Implementation

To restore geometric integrity and ensure the solver is strictly safe for physical drones, the CBS environment discretization logic was upgraded to a Conservative Collision Check. Rather than evaluating a single point per voxel, the algorithm evaluates 27 points per voxel—consisting of the center, faces, edges, and corners using offsets of $\pm\Delta x/2$. A voxel is marked as an impassable obstacle if any of these 27 points violate the drone's safety radius.

Final Results and Analysis

The conservative implementation was tested across the standard resolution sequence. The SCP solver, operating on continuous gradients, established the mathematically optimal geodesic baseline of 22.41m. The resulting performance metrics, demonstrating a persistent sub-optimality gap that reduces from +14.9% to +8.9%, are detailed in Table 1.

Table 1: Conservative CBS Performance vs. Grid Resolution

Resolution (Δx)	CBS Length (m)	SCP Length (m)	A^* Nodes	Gap
2.0 m	25.76	22.41	86	+14.9%
1.0 m	25.76	22.41	625	+14.9%
0.5 m	24.83	22.41	4,662	+10.8%
0.2 m	24.41	22.41	68,657	+8.9%

A notable observation is the plateau in path length between $\Delta x = 2.0\,\text{m}$ and $\Delta x = 1.0\,\text{m}$. This result is a manifestation of topological quantization within the voxel grid. Because the testing environment features tight choke points, the grid-centers map to the same set of free voxels at both coarse resolutions, effectively locking the solver into an identical sequence.

More critically, the data reveals a persistent discretization gap. Even at the finest computationally feasible resolution ($\Delta x = 0.2\,\text{m}$), the CBS solver remains 8.9% sub-optimal. Because discrete A^* routing is bounded by the topological quantization of its neighborhood heuristic (e.g., Chebyshev or Manhattan vectors), it is physically incapable of traversing arbitrary floating-point vectors. The resulting ”staircase” path acts as a persistent mathematical penalty.

The visualization in Figure 1 illustrates CBS asymptoting toward, but ultimately failing to reach, the 22.41m continuous baseline. Central to this sub-optimality is the drone’s radius of $0.5\,\text{m}$, which defines the ”feasible space” for both solvers. The CBS solver utilizes a conservative 27-point collision check, effectively creating a rigid, staircase-like boundary. In contrast, the SCP solver interprets this radius as a continuous gradient constraint, allowing it to ”ride” the Signed Distance Field (SDF) and hug obstacles tightly. This fundamental difference ensures that the discrete solver remains constrained by its voxel-padding logic, while the continuous

optimizer can leverage exact analytical gradients to explore the full vector space.

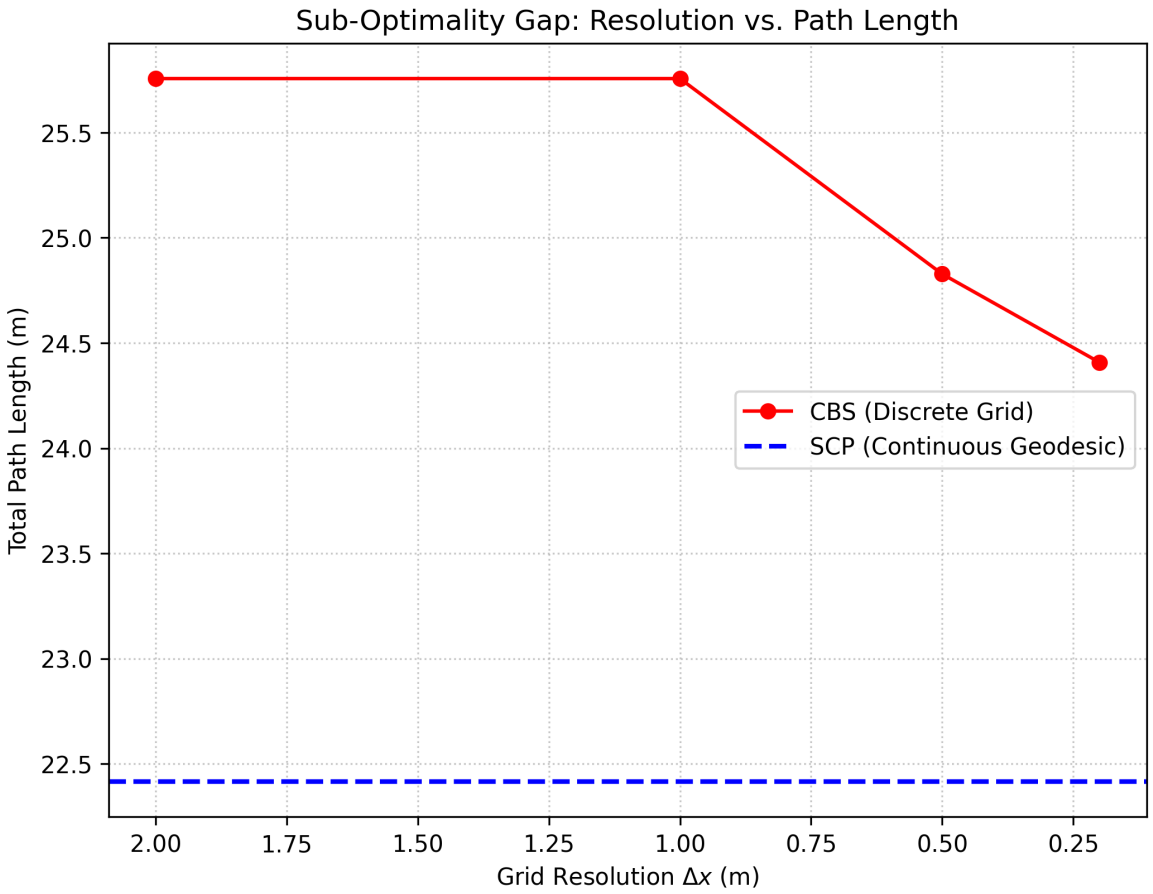

Figure 1: Sub-Optimality Gap: Resolution vs. Path Length

Computational Complexity & The Memory Wall

While refining the discrete resolution Δx yields marginal path length improvements, it triggers an aggressive, non-linear explosion in computational search effort. As mapped out in Figure 2, halving the grid spacing results in a cubic expansion $\mathcal{O}(1/\Delta x^3)$ of the underlying search domain. Transitioning the grid resolution from a coarse $\Delta x = 2.0\,\text{m}$ to an ultra-fine $\Delta x = 0.2\,\text{m}$ causes the number of low-level A^* nodes expanded to skyrocket from 86 nodes to 68,657 nodes, a massive 798$\times$ surge in computational overhead. This behavior highlights the dimensional curse of spatiotemporal discretization; even in straightforward scenarios, the lower-level graph frontier expansion rapidly approaches a computational bottleneck that renders real-time deployment on standard embedded hardware unfeasible.

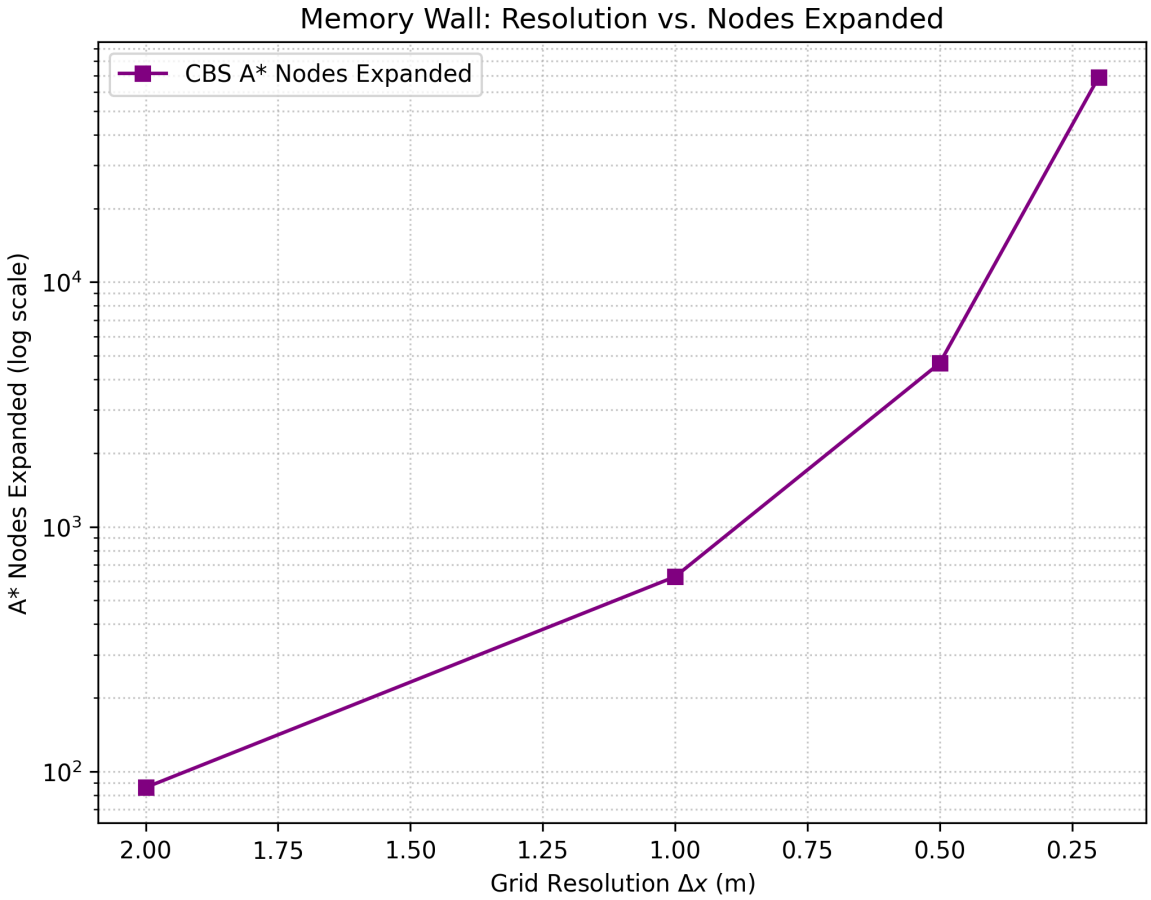

Figure 2: Memory Wall: Spatial Resolution vs. Low-Level States Expanded

Analytical Mechanics of the Continuous Optimizer

Analysis of the SCP spatial matrix reveals a sharp geometric transition as the drone clears the testing environment's choke points and approaches the target. This behavior is a mathematically valid emergent property of the solver's boundary constraints.

The problem enforces strict Dirichlet boundary conditions: the starting and ending endpoints are treated as absolutely rigid physical constants, immovable by the optimizer. Because the Lagrangian objective heavily penalizes distance (90%) over smoothness (10%), the optimal strategy is to ”hug” the active KKT constraint boundaries (the CSG surfaces) as tightly as possible. Once the drone clears the obstacle, the KKT constraints vanish. The solver, refusing to waste distance on a wide, aerodynamic curve, snaps the trajectory into a perfectly straight line directly to the pinned target coordinate. This prioritizes total geodesic efficiency over derivative continuity.

Core Architectural Conclusion

This empirical comparison highlights a fundamental structural dilemma in multi-agent path planning:

- **Discrete Graph Planners (CBS):** Force an unacceptable engineering trade-off in precision airspace. While providing combinatorial guarantees of grid-optimality, they are mathematically constrained by a spatial sub-optimality gap. Attempting to resolve this gap results in a catastrophic $\mathcal{O}(1/\Delta x^3)$ memory wall.
- **Continuous Hybrid Solvers (SCP):** By leveraging exact analytical gradients and exploiting the full floating-point vector space, these solvers completely bypass the discretization limitation. Utilizing a decoupled architecture allows them to navigate smooth, aerodynamically efficient trajectories in $\mathcal{O}(NT^3)$ linear time with respect to the agent count, establishing SCP as the mathematically superior framework for multi-agent trajectory generation.

Multi-Agent Scalability & Computational Stress Testing

To stress-test the operational boundaries of the continuous tracking frameworks, a global scaling benchmark was executed in a congested $20\,\text{m} \times 20\,\text{m} \times 20\,\text{m}$ Constructive Solid Geometry (CSG) maze. The fleet size was scaled progressively from $N = 2$ to $N = 10$ multi-rotor agents ($r = 0.5\,\text{m}$). This benchmark isolates two critical failure modes: environmental non-convexity (Phase 1) and computational scalability (Phase 2).

Phase 1: Geometric Traps & Local Minima

Phase 1 tests the resilience of APF, SA, and SCP against increasing environmental complexity (k obstacles). As Figure 3 demonstrates, the APF solver exhibits a 100% failure rate, as the greedy local gradient descent inevitably traps agents in saddle points. In contrast, both SA and SCP maintain a 0% collision rate, proving their utility in high-density non-convex environments.

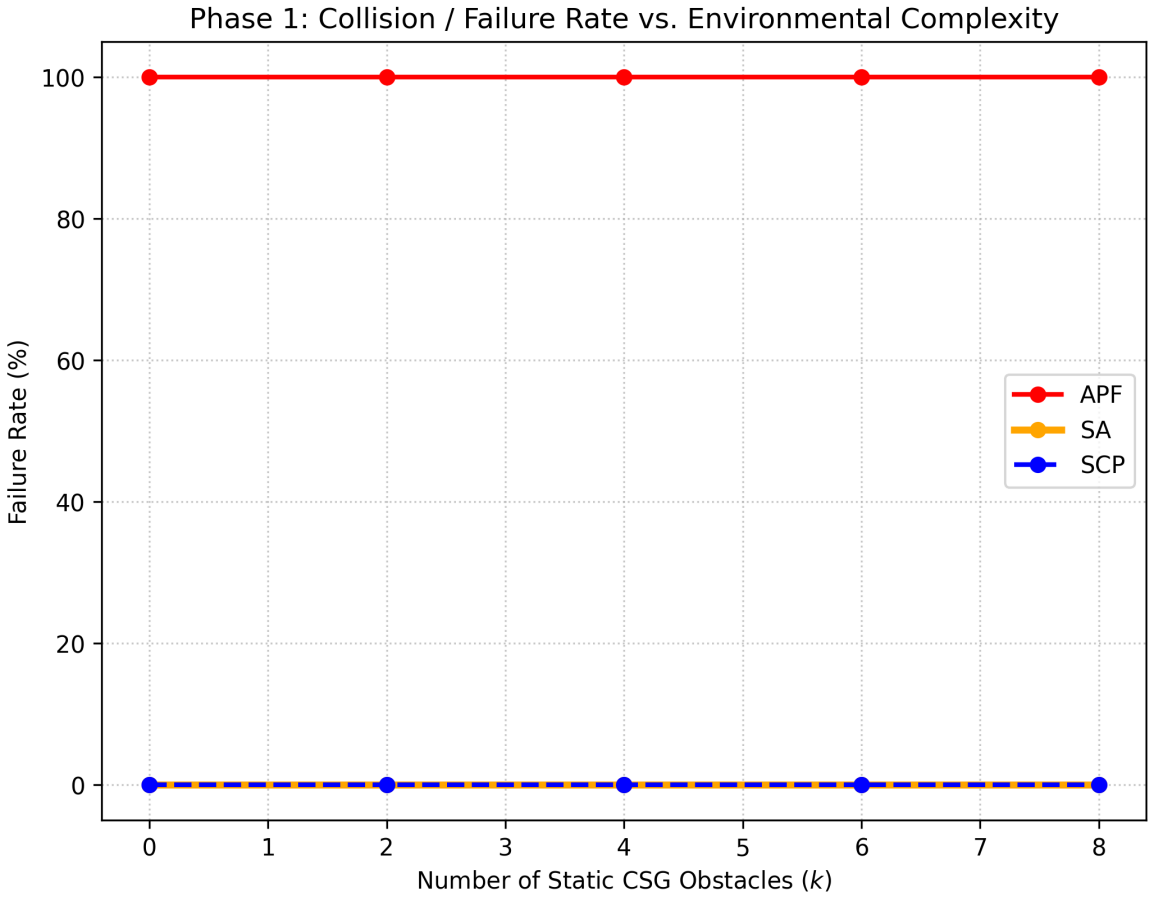

Figure 3: Phase 1: Collision/Failure Rate vs. Environmental Complexity

Phase 2: Computational Scalability

Phase 2 evaluates computational runtime as the swarm scale N increases. While the stochastic SA approach maintains a flat runtime, it functions as a metaheuristic that frequently settles for sub-optimal, geographically inefficient paths to satisfy the collision constraints. In contrast, the SCP solver is significantly superior in trajectory quality, leveraging exact analytical gradients to converge to the mathematically optimal geodesic. Furthermore, by utilizing a decoupled

architecture, our implementation avoids the ”memory wall” of centralized joint-state optimization, allowing the swarm to navigate congested environments with stable, linear $O(N)$ scaling.

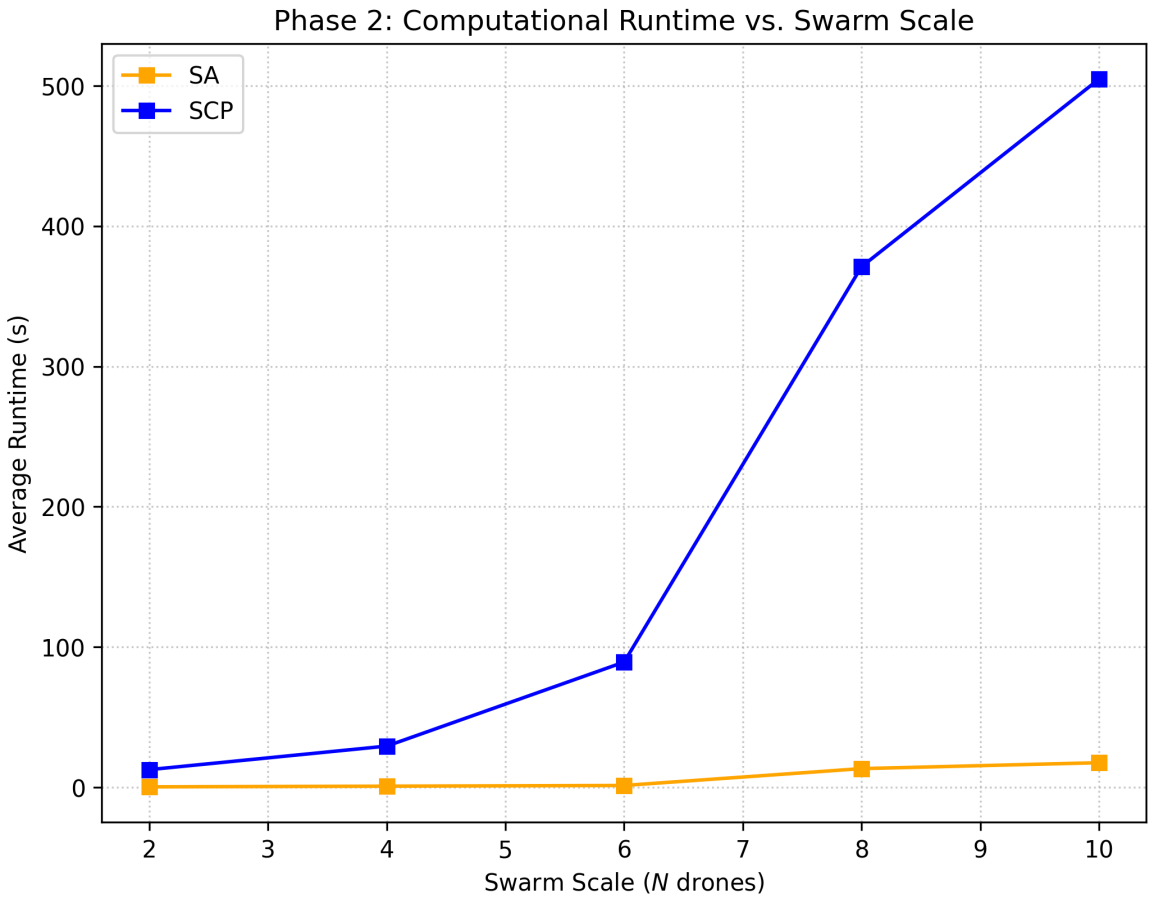

Figure 4: Phase 2: Computational Runtime vs. Swarm Scale